Prof. Ashwani Mishra
Assistant Professor
CSE Department

1. Preparing the Application for Deployment

What is Deployment?

Deployment means making the application available on the internet so users can access it.

Example:

Before deployment

http://localhost:4200

After deployment

https://studentapp.com

Steps Before Deployment

Checklist:

1 Fix bugs
2 Remove development code
3 Optimize application
4 Configure environment variables
5 Build production version

Angular Production Build

Run command:

ng build --configuration production

This creates a **dist folder** containing optimized files.

Example structure:

Ashwani Mishra

```
dist/
  index.html
  main.js
  styles.css
```

Node.js Production Setup

Install dependencies:

```
npm install
```

Start server:

```
node server.js
```

Production servers usually use process managers like PM2.

Install PM2:

```
npm install pm2 -g
```

Run server:

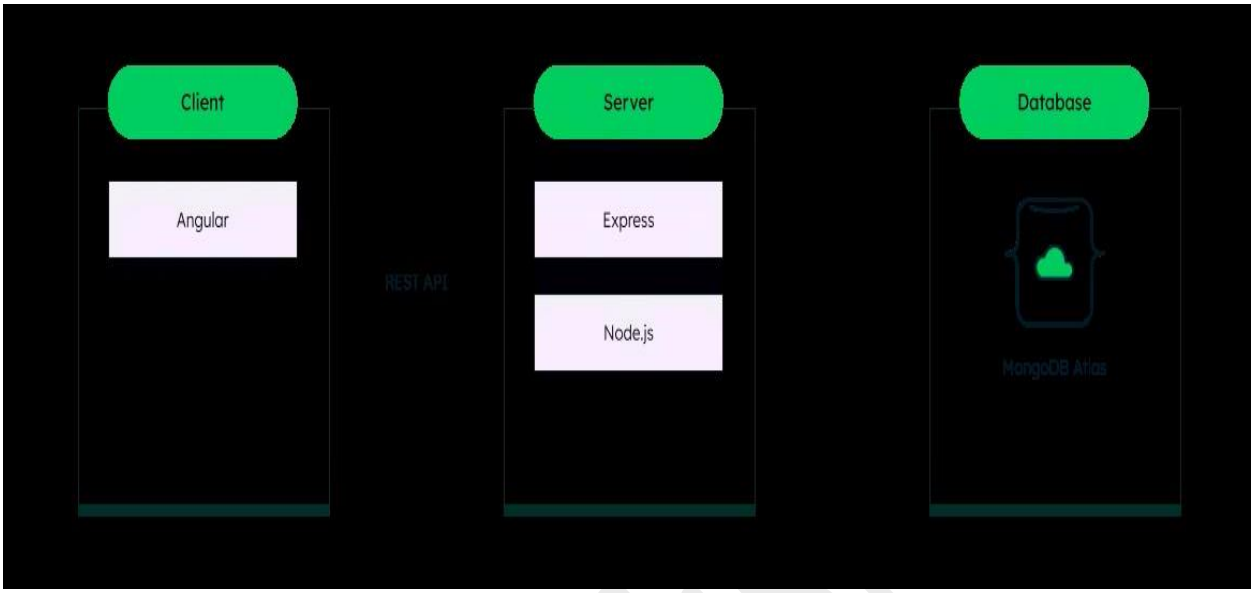
```
pm2 start server.js
```

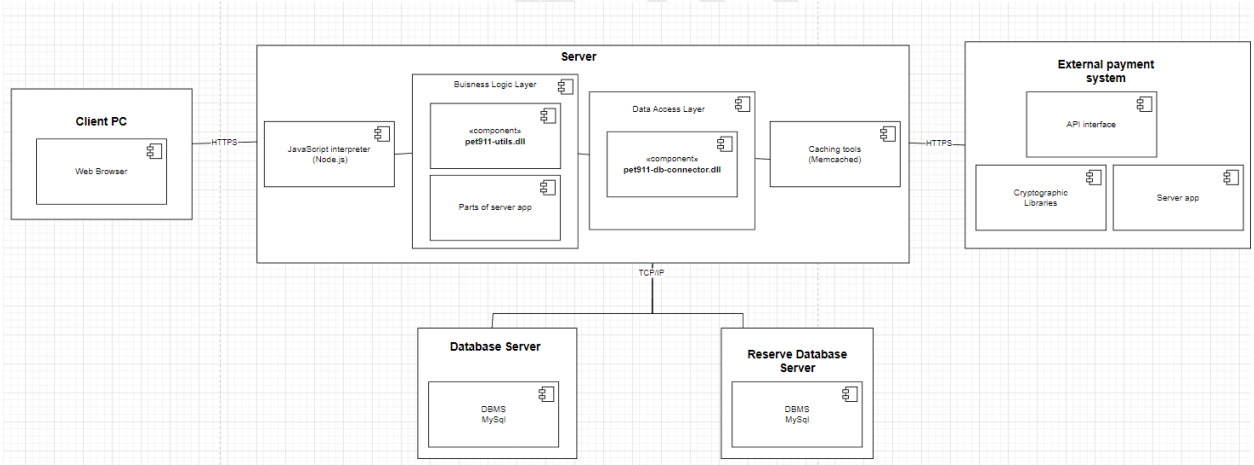
PM2 advantages:

- Auto restart server
- Monitor performance
- Background execution

Deployment Architecture

Ashwani Mishra





Flow:

User Browser → Web Server (Angular Build) → Node.js API Server →

MongoDB Database

---

2. Hosting and Server Setup Options

Applications can be hosted on different platforms.

Hosting Options:

Platform	Usage
AWS	Cloud hosting
Google Cloud	Cloud infrastructure
DigitalOcean	VPS hosting
Heroku	Easy deployment
Vercel / Netlify	Frontend hosting

Example Deployment Structure:

Frontend (Angular) → Netlify / Vercel → Backend (Node.js) → AWS / DigitalOcean

↓

MongoDB Atlas <-- Database

MongoDB Cloud (Atlas)

MongoDB Atlas provides cloud database hosting.

Example connection string:

mongodb+srv://user:password@cluster.mongodb.net/mydb

Use in Node.js:

mongoose.connect("mongodb+srv://cluster-url")

Domain and Server Setup

Steps:

1 Buy domain (example: studentapp.com)
2 Configure DNS
3 Connect domain to server
4 Enable SSL certificate

Example DNS flow

Ashwani Mishra

Domain → DNS → Hosting Server → Application

---

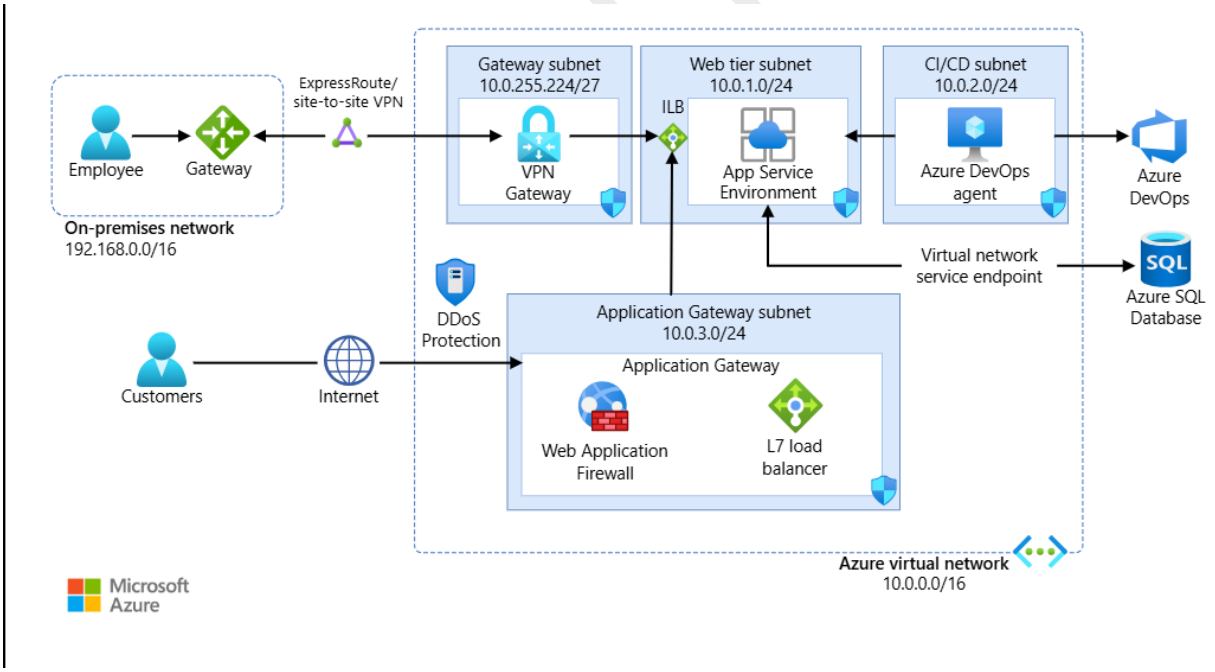
3. Security Best Practices

Security protects the application from hackers and attacks.

Common web attacks:

Attack	Description
SQL Injection	Database manipulation
XSS	Injecting malicious scripts
CSRF	Unauthorized requests
Brute Force	Password attacks

Security Architecture



Security Best Practices

1. Use HTTPS

HTTPS encrypts communication.

Example:

https://example.com

2. Use JWT Authentication

Example:

```
const token = jwt.sign(user,"secret")
```

3. Password Hashing

Never store plain passwords.

Example using bcrypt:

```
const bcrypt = require("bcrypt")

bcrypt.hash(password,10)
```

4. Input Validation

Example:

```
if(!email.includes("@")){
 return res.status(400).send("Invalid Email")
}
```

5. Use Helmet Middleware

Install:

```
npm install helmet
```

Use:

```
const helmet = require("helmet")
app.use(helmet())
```

Helmet secures HTTP headers.

Ashwani Mishra

4. Performance Optimization

Optimization improves **speed and efficiency of applications**.

Performance Optimization Techniques

Technique	Description
Caching	Store frequently used data
Compression	Reduce file size
Lazy loading	Load modules when needed
Database indexing	Faster queries

Angular Lazy Loading

Example:

```
{
 path:'admin',
 loadChildren:()=>import('./admin/admin.module')
 .then(m=>m.AdminModule)
}
```

Node.js Compression

Install:

```
npm install compression
```

Use:

```
const compression = require("compression")
app.use(compression())
```

This reduces response size.

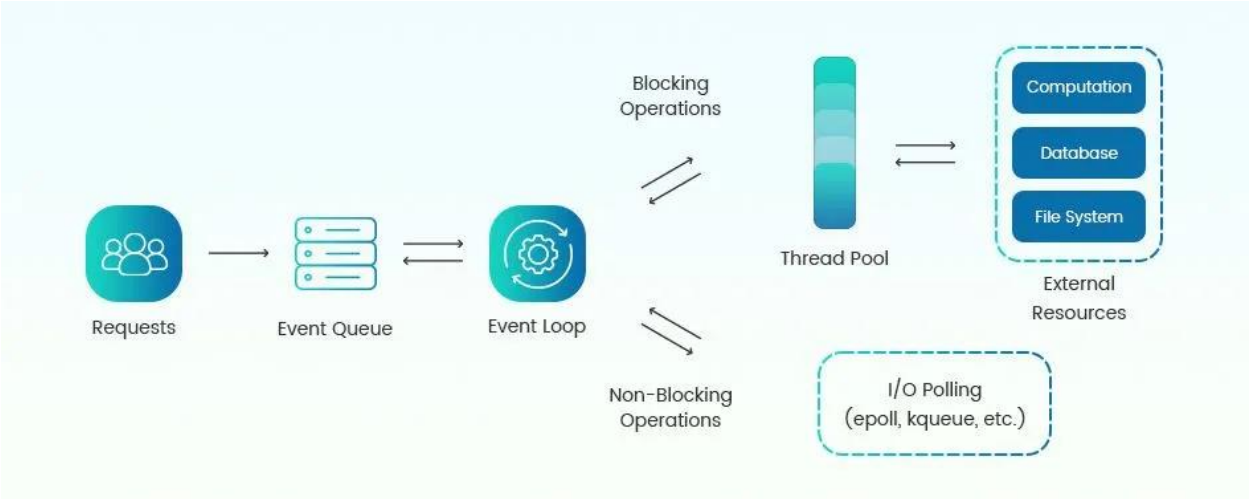
Ashwani Mishra

Database Indexing Example

db.students.createIndex({name:1})

Improves query speed.

Performance Optimization Diagram



---

5. Testing Before Deployment

Testing ensures the application works properly.

Types of Testing:

Testing Type	Purpose
Unit Testing	Test single component
Integration Testing	Test API integration
System Testing	Test complete system
Load Testing	Test performance

Angular Testing

Tools:

- Jasmine
- Karma

Example test:

```
it("should create component",()=>{
 expect(component).toBeTruthy()
})
```

Backend Testing

Tools:

- Mocha
- Chai
- Postman

Example API test:

GET /api/students

Expected response:

```
[
 { "name":"Rahul" }
]
```

---

6. Version Control

Version control tracks **code changes over time**.

Most popular tool:

Git

Platforms:

Ashwani Mishra

- GitHub
- GitLab
- Bitbucket

Git Workflow

Developer → Git Repository → Commit → Push → Team Collaboration

Basic Git Commands

Initialize repository:

```
git init
```

Add files:

```
git add .
```

Commit changes:

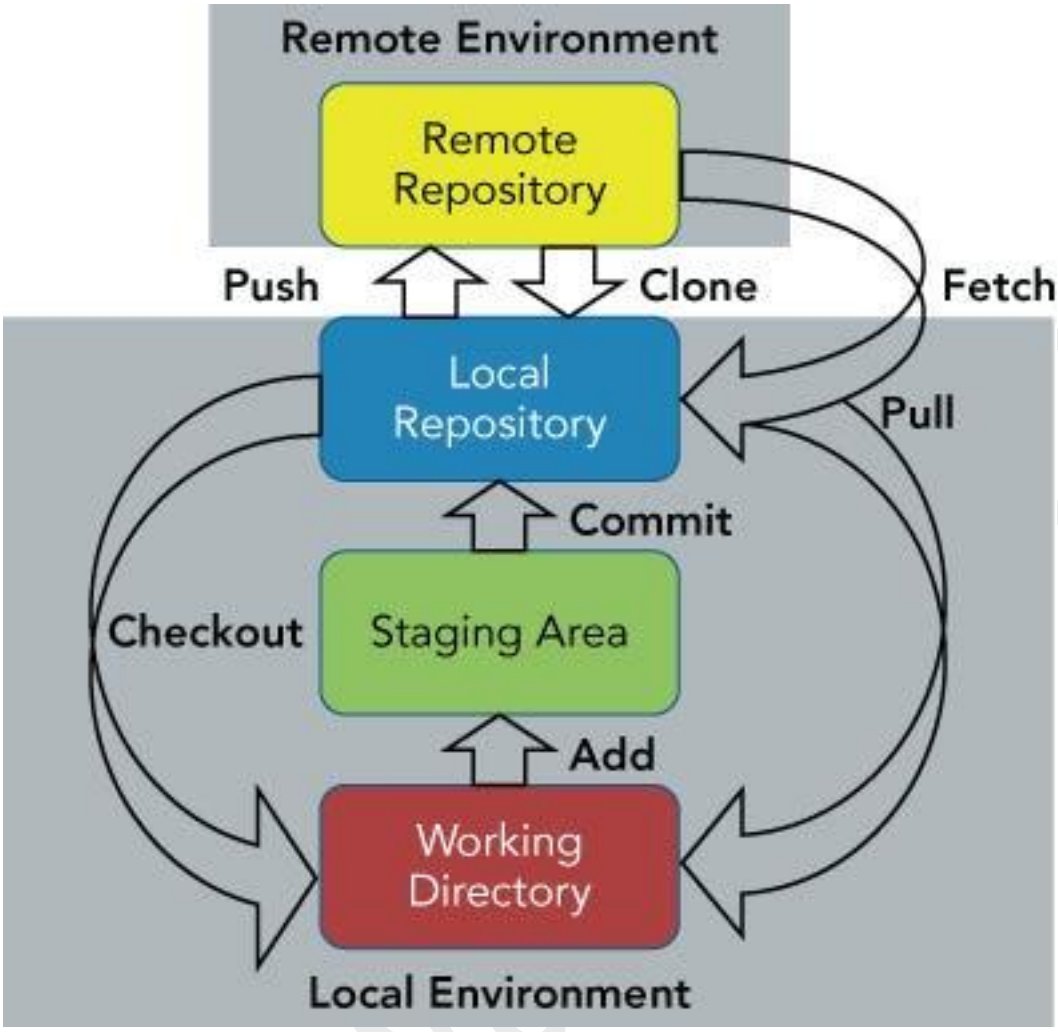
```
git commit -m "Initial commit"
```

Push to GitHub:

```
git push origin main
```

Version Control Diagram

Ashwani Mishra



---

7. Continuous Integration (CI)

CI automatically **builds and tests code whenever changes are pushed**.

Example tools:

Tool	**Platform**
GitHub Actions	GitHub
Jenkins	CI server
GitLab CI	GitLab
CircleCI	Cloud CI

CI Workflow:

Developer Push Code → Git Repository → CI Server → Run Tests

↓

Deploy ← Build Application

Complete MEAN Stack Production Flow

Angular Frontend

↓

Hosted on Netlify / Vercel

↓

Node.js Express API

↓

Cloud Server (AWS / DigitalOcean)

↓

MongoDB Atlas

Ashwani Mishra